AN0672

Firmware Converter Guide

Introduction

The Firmware Converter is a python tool that is distributed with the Cirrus Logic MCU Driver SDK. This tool can be used to convert Cirrus Logic firmware and tuning files into source code that can be included in MCU driver projects.

The tool processes .wmfw and .bin files to create .c and .h source that gives MCU Drivers access to the data that needs to be loaded into the DSP, as well as symbols that represent important firmware registers.

Table of Contents

1 Overview

Cirrus Logic provides firmware and tuning files in .wmfw and .bin file formats.

Firmware files in the .wmfw format contain executable program data, DSP memory blocks, and a map of the externally-visible firmware registers. A firmware file is specific to one part and revision, but may be re-used on multiple projects with that part/revision.

Tuning files in the .bin file format, also referred to as WMDR format, contain project-specific data that configures and tunes the algorithms in the firmware. Cirrus may provide 'generic' tunings to evaluate a system before it is characterized, but generally tunings should only be used on the platform they are targeting. The data in tuning files is located with 'relative' addressing, so any software that consumes tunings must use a .wmfw to determine the absolute addresses of tuning data.

The contents of the .wmfw and .bin files are illustrated in the following figure.

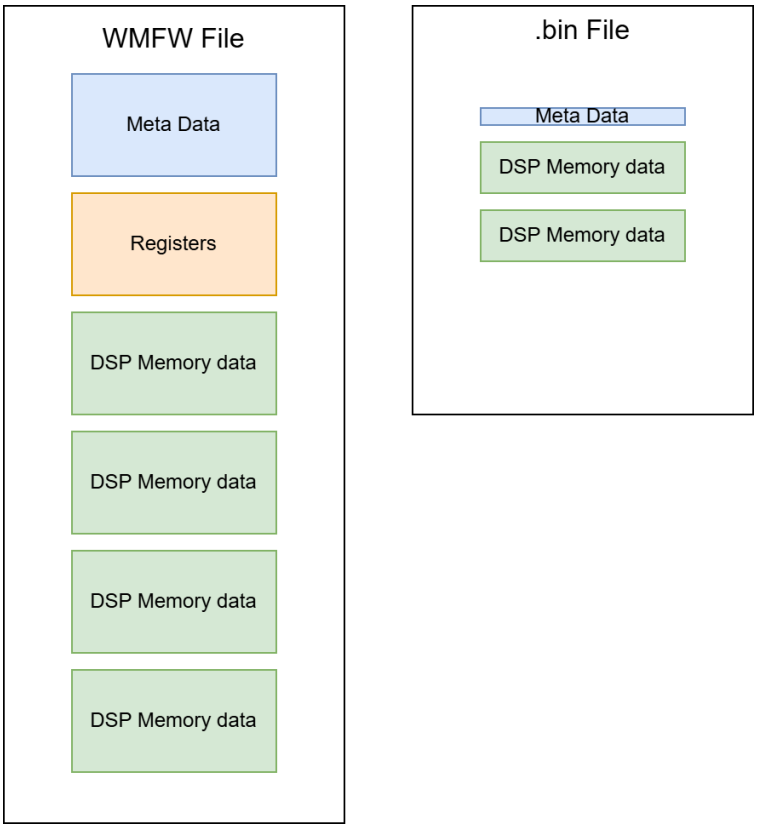


Figure 1 Firmware (.wmfw) and Tuning (.bin) File Contents

The firmware and tuning files are portable to many different systems using Cirrus parts. Because embedded platforms can be resource-constrained or lack filesystems, the .wmfw and .bin files must be converted to source code or binary data in order to be integrated on some microcontroller devices.

The conversion to source code and header files is illustrated in the following figure.

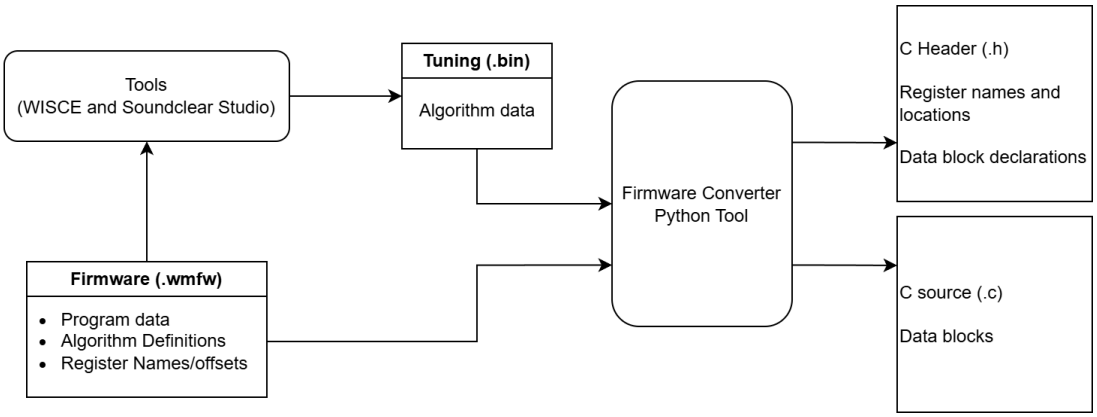


Figure 2 Firmware Converter Input/Output

2 Using the Firmware Converter Tool

Prerequisites for using the firmware converter tool are as follows:

- The tool requires Python 3.6+. The tool is not compatible with Python 2.x or versions earlier than Python 3.6.
- The Cirrus Logic MCU Driver SDK can be found on Github at: https://github.com/CirrusLogic/mcu-drivers
- The tool is located in the "tools/firmware_converter/" directory.

Example usage of the tool is shown in the following code example:

```
firmware_converter.py  [Output format] [Device] [Firmware] [Optional arguments]
```

The tool requires three arguments:

- Output format – See below for further details. Valid options for MCU integration include "export" and "fw_img_v2".
- Device – Which Cirrus Logic part is being used.
- Firmware – Path to a .wmfw RAM firmware file or 'ROM' to use the device ROM firmware.

The tool also supports optional arguments:

- Tuning files - The *–-wmdr* option can be used to specify one or more .bin files to be converted with the firmware.
- Output directory - The *--output-directory* option can be used to specify a directory for output source files. MCU Driver SDK sample applications expect the converted files to exist in the 'fw' directory for each part.
- Preserve filename - The *--preserve-filename* option can be used to keep the filename when creating names for data blocks in .bin files instead of generic names (coeff_0, coeff_1, ...).
- Binary Output – The *--binary-output* option can be used to create a binary file with only the converted data and no C code wrapper.

Typical use cases for the tool are illustrated with the following code examples:

ROM FW + Wavetable

```
python3 ../../tools/firmware_converter/firmware_converter.py export cs40l50 ROM --wmdr
cs40l50/fw/cs40l50_wt.bin
```

RAM FW + Wavetable

```
python3 ../../tools/firmware_converter/firmware_converter.py export cs40l50
cs40l50/fw/CS40L50_Rev3.4.5.wmfw --wmdr cs40l50/fw/cs40l50_wt.bin
```

RAM FW + Wavetable + Tuning File

```
python3 ../../tools/firmware_converter/firmware_converter.py export cs40l50
cs40l50/fw/CS40L50_Rev3.4.5.wmfw --wmdr cs40l50/fw/cs40l50_wt.bin cs40l50/fw/cs40l50_tuning.bin
```

The tool supports different formats for the output data. These are selected using the 'output format' argument:

- 'print' - Print the data out to the console and do not make any files
- 'json' - Create a JSON representation of the data
- 'wisce' - Create a script that can be used by WISCE
- 'export' - Create C source files with the data exported to be compiled into a MCU application
- 'fw_img_v2' - Create C source files with the data in a self-contained, portable image

The two options used for MCU driver integration are 'export' and 'fw_img_v2'.

The 'export' format is simple and can be used in many applications. When using this format, the data is compiled into the MCU driver.

The 'fw_img_v2' format is more complex and requires driver code from the SDK to use. This format is more portable and can enable storing the firmware/tuning data in an external memory. When stored separately, the firmware and tuning can be updated without rebuilding the MCU driver.

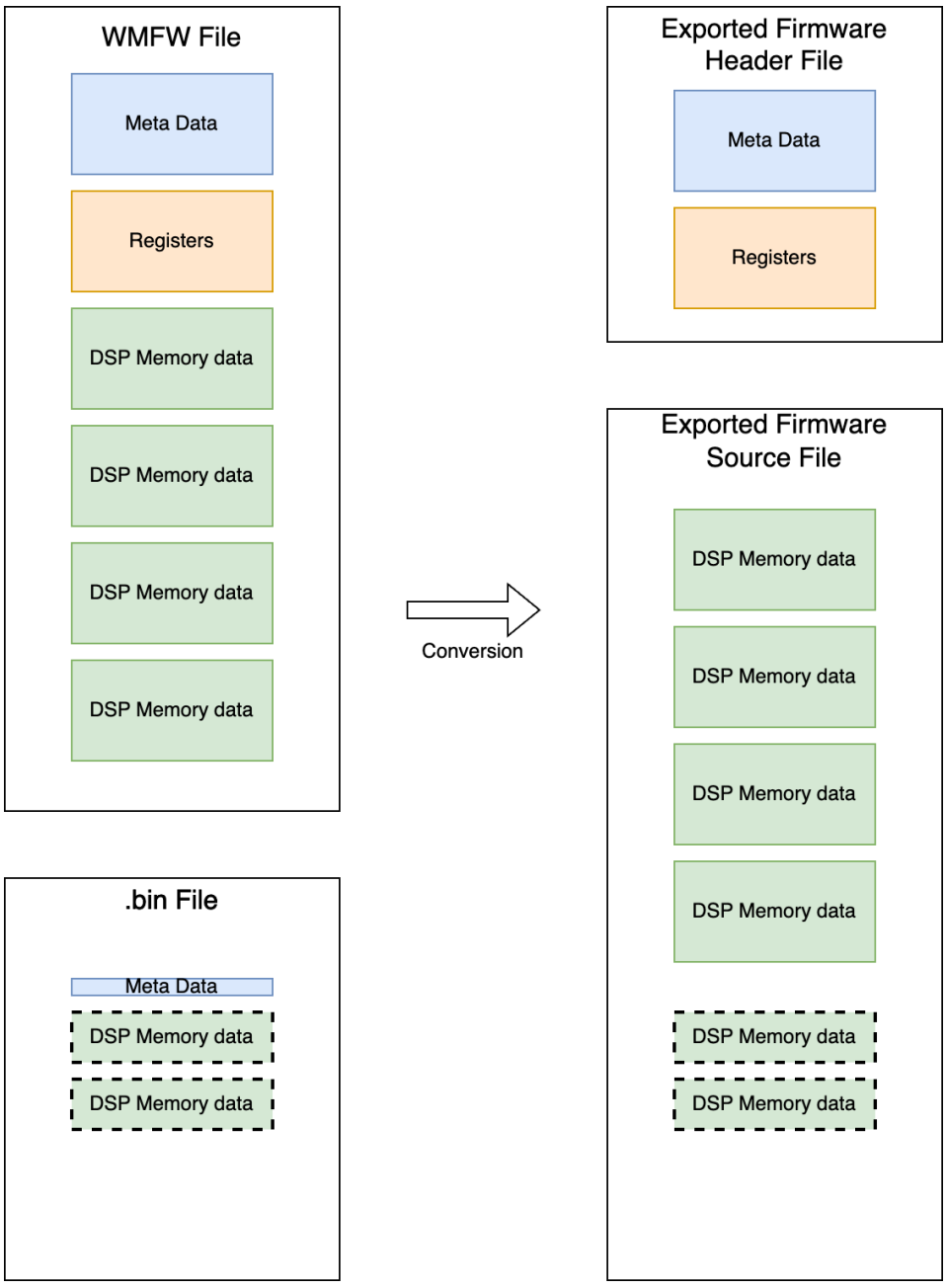
3 Output Files - export format

The "export" output format creates .c and .h files that contain separate data arrays for each data block and define register addresses for all named fields exposed in the firmware.

The files are created in the output directory:

- ***_firmware.h** – Header file containing register address #defines, and data block declarations
- ***_firmware.c** – Source file containing data block definitions

The following figure shows the .wmfw and .bin files contents, compared with the header and source files created using the export format.



3.1 Output Files Example

An example output is shown for exporting the CS40L50 RAM firmware with one wavetable .bin file. This creates the output files "cs40l50_firmware.h" and "cs40l50_firmware.c".

Command (run from the 'cs40l50/fw/' directory):

```
../../tools/firmware_converter/firmware_converter.py export cs40l50 CS40L50_Rev3.4.8.wmfw --
wmdr cs40l50_wt.bin
```

Excerpt from cs40l50_firmware.h:

```
#define FIRMWARE_CS40L50_HALO_STATE 0x28021E0

#define cs40l50_total_fw_blocks (22)

#define cs40l50_total_coeff_blocks_0 (6)

typedef struct
{
    uint32_t block_size;    ///< Size of block in bytes
    uint32_t address;       ///< Control Port register address at which to begin loading
    const uint8_t *bytes;   ///< Pointer to array of bytes consisting of block payload
} halo_boot_block_t;

extern const halo_boot_block_t cs40l50_fw_blocks[];

extern const halo_boot_block_t cs40l50_coeff_0_blocks[];
```

Excerpt from cs40l50_firmware.c:

```
const uint8_t cs40l50_fw_block_0[] = { 0x00,0x04,0x00,0x00,0x00, ... };
const uint8_t cs40l50_fw_block_1[] = { 0x00,0xAB,0xCD,0xEF,};
...

const halo_boot_block_t cs40l50_fw_blocks[] = {
    {
        .address = 0x02000000,
        .block_size = 348,
        .bytes = cs40l50_fw_block_0
    },
    {
        .address = 0x02800180,
        .block_size = 4,
        .bytes = cs40l50_fw_block_1
    },
...
};
```

3.2 Driver Loading Example - export

Driver loading examples for the export and fw_img_v2 formats can be found in cs40l50/bsp/bsp_cs40l50_baremetal.c in the bsp_dut_boot() function.

Example from cs40l50/bsp/bsp_cs40l50_baremetal.c

```
uint32_t bsp_dut_boot(void)
{

    uint32_t ret;
    int i;
    for (i = 0; i < cs40l50_total_fw_blocks; i++) {
        ret = regmap_write_block((&cs40l50_driver.config.bsp_config.cp_config),
                                 cs40l50_fw_blocks[i].address,
                                 (uint8_t *)cs40l50_fw_blocks[i].bytes,
                                 cs40l50_fw_blocks[i].block_size);
        if (ret == CS40L50_STATUS_FAIL)
        {
            return BSP_STATUS_FAIL;
        }
    }


    for (i = 0; i < cs40l50_total_coeff_blocks_0; i++) {
        ret = regmap_write_block((&cs40l50_driver.config.bsp_config.cp_config),
                                 cs40l50_coeff_0_blocks[i].address,
                                 (uint8_t *)cs40l50_coeff_0_blocks[i].bytes,
                                 cs40l50_coeff_0_blocks[i].block_size);
        if (ret == CS40L50_STATUS_FAIL)
        {
            return BSP_STATUS_FAIL;
        }
    }

    regmap_write((&cs40l50_driver.config.bsp_config.cp_config), CS40L50_DSP1_CCM_CORE_CONTROL,
0x00000281);
```

The driver loops over all the blocks in the fw and coeff arrays. Each block is written to the device using the address, size, and data contained in the halo_boot_block_t element.

4 Output Files - fw_img_v2 format

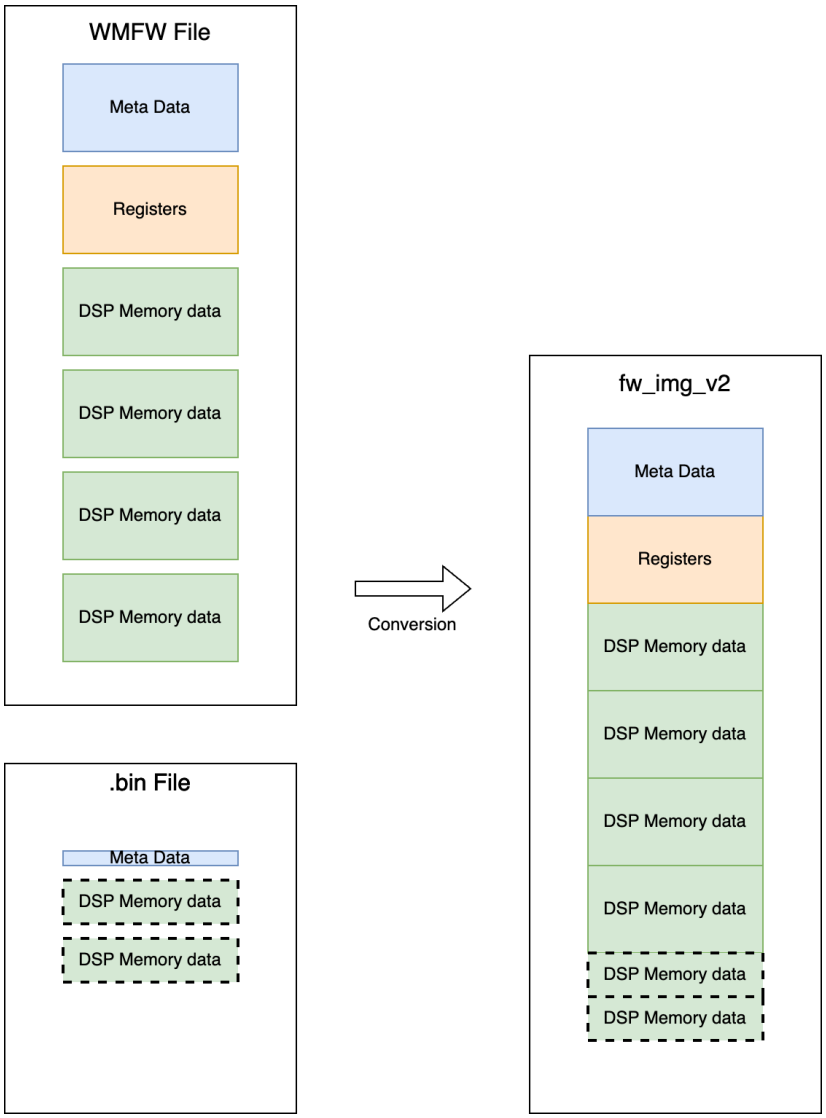
The "fw_img_v2" output format creates .c and .h files that contain a single contiguous data array with all the data and information contained in the firmware and tuning(s). The array must be parsed by additional SDK code in order to locate sections that need to be loaded into the device.

This format has advantages if the firmware and tuning data needs to be stored separately from the application processor. Although there is extra driver code required to parse the firmware image, the driver does not need to compile any literal symbols associated with the firmware (addresses, data arrays, sizes) so the firmware and tunings can be updated after the driver software has been frozen. The primary way this is supported is register access through the symbol linking table rather than through absolute addresses.

The files are created in the output directory:

- ***_fw_img.h** – Firmware image header file
- ***_fw_img.c** – Firmware image source file

The following figure shows the .wmfw and .bin file contents, compared with the image created using the fw_img_v2 format.



The following sections contain example output for converting the CS40L26 RAM firmware using the fw_img_v2 format with no additional .bin files. This creates the output files "cs40l26_fw_img.c" and "cs40l26_fw_img.h".

Command:

```
python3 tools/firmware_converter/firmware_converter.py fw_img_v2 cs40l26
cs40l26/fw/CS40L26_Rev7.2.24.wmfw
```

4.1 Firmware Image Header (*_fw_img.h)

The firmware image header file (*_fw_img.h) contains an extern declaration for the data in the firmware image source file (*_fw_img.c). This header should be included by the BSP layer that is responsible for loading the firmware data.

Excerpt from cs40l26_fw_img.h:

```
extern const uint8_t cs40l26_fw_img[];
```

All of the data and information is contained in the single data array cs40l26_fw_img[]. This data must be handled using the fw_img library code in the SDK.

4.2 Firmware Image Source (*_fw_img.c)

The firmware image source file contains a contiguous byte array of data that should be interpreted by the driver to load the DSP memory.

The tool annotates the data with comments delineating each section:

- Header – Provides information to the driver about how to parse the image
- Symbol Linking Table – Enables DSP register access
- Algorithm ID list – Provides information to the driver about the features in the firmware
- Firmware Data – DSP data sections from the converted .wmfw or .bin files
- Footer – Marks the end of the image

Excerpt from cs40l26_fw_img.c:

```
const uint8_t cs40l26_fw_img[] = {
    // Header
    ...
    // Symbol Linking Table
    ...
    // Algorithm ID List
    ...
    // Firmware Data
    ...
    // Footer
    ...
};
```

All data is little-endian; the least significant byte for all IDs, sizes, and addresses occurs first in the 32-bit word. Little-endian data is shown in the following examples.

```
// Symbol Linking Table
0x02,0x00,0x00,0x00, // FIRMWARE_CS40L50_HALO_STATE = 0x00000002
0xE0,0x21,0x80,0x02, // 0x28021e0
...
// Firmware Data
0x5C,0x01,0x00,0x00, // FW_BLOCK_SIZE_0 = 0x0000015c
0x00,0x00,0x00,0x02, // FW_BLOCK_ADDR_0 = 0x02000000
```

4.2.1 Header

The Header defines the size of the subsequent sections, the version of the firmware that was parsed, and the version of the converter tool.

```
// Header
0xFF,0x98,0xB9,0x54, // IMG_MAGIC_NUMBER_1
0x02,0x00,0x00,0x00, // IMG_FORMAT_REV
0x74,0x6A,0x00,0x00, // IMG_SIZE
0xAC,0x02,0x00,0x00, // SYM_TABLE_SIZE
0x0F,0x00,0x00,0x00, // ALG_LIST_SIZE
0xD4,0x00,0x18,0x00, // FW_ID
0x18,0x02,0x07,0x00, // FW_VERSION
0x1F,0x00,0x00,0x00, // DATA_BLOCKS
0x2C,0x10,0x00,0x00, // MAX_BLOCK_SIZE
0x00,0x00,0x00,0x00, // FW_IMG_VERSION
```

The contents of the header are described in the following table.

Field	Description
IMG_MAGIC_NUMBER_1	Tools can validate the first word
IMG_FORMAT_REV	version of the format (v2)
IMG_SIZE	size of the image
SYM_TABLE_SIZE	size of the sym table
ALG_LIST_SIZE	size of the alg list
FW_ID	firmware ID
FW_VERSION	firmware version
DATA_BLOCKS	number of data blocks
MAX_BLOCK_SIZE	maximum block size
FW_IMG_VERSION	image minor version

Definition from common/fw_img.c

```
/**
 * Header for fw_img_v2
 */
typedef struct
{
    uint32_t img_size;
    uint32_t sym_table_size;
    uint32_t alg_id_list_size;
    uint32_t fw_id;
    uint32_t fw_version;
    uint32_t data_blocks;
    uint32_t max_block_size;
    uint32_t fw_img_release;
} fw_img_v2_header_t;
```

4.2.2 Algorithm ID List

The Algorithm ID List provides an ID for each algorithm present in the firmware image. A unique ID is assigned for each feature in the firmware.

A driver can use the Algorithm ID List to verify compatibility between the firmware image and the application.

4.2.3 Symbol Linking Table

The Symbol Linking Table contains pairs of IDs (defined in the *_sym.h file) and addresses of DSP registers.

```
// FIRMWARE_CS40L26
#define CS40L26_SYM_FIRMWARE_HALO_STATE                         (0x2)
#define CS40L26_SYM_FIRMWARE_HALO_HEARTBEAT                     (0x3)
```

HALO_STATE and HALO_HEARTBEAT are given the IDs 2 and 3 in cs40l26_sym.h

```
// Symbol Linking Table
0x02,0x00,0x00,0x00, // FIRMWARE_HALO_STATE
0xA8,0x0F,0x80,0x02, // 0x2800fa8
0x03,0x00,0x00,0x00, // FIRMWARE_HALO_HEARTBEAT
0xAC,0x0F,0x80,0x02, // 0x2800fac
```

The symbol linking table provides the absolute addresses of HALO_STATE and HALO_HEARTBEAT in the DSP X memory. See Section 4.3 for further details on Register Access.

4.2.4 Firmware Data

The Firmware Data contains the converted data blocks from the provided .wmfw or .bin files. Firmware data blocks consist of a size, a destination address, and bulk data to be transferred to the DSP. The data is often 'packed' for efficient storage (DSP words are 24 or 40 bits) and not meant to be human-readable. Whenever possible, the MCU driver should avoid breaking up a firmware data block into smaller sections.

4.2.5 Footer

The Footer marks the end of the image. The last word of the image is a 32-bit Fletcher checksum of the image data.

4.3 Binary Output

The fw_img_v2 data can be output into a binary file that contains only the byte array data without any C code wrapping it. To create the binary file, add the option '*--binary-output*' when running the conversion. The .c and .h files will still be created in addition to the binary file.

4.4 Register Access fw_img_v2 - Symbol IDs

When using the fw_img_v2 data format, registers are accessed by looking up the absolute address in the Symbol Linking Table, using an ID that is shared by the firmware image and the MCU application.

The absolute address of a register may change depending on the .wmfw or .bin version. When using the fw_img_v2 format, the driver does not need to be rebuilt if the firmware or tuning is updated. Because the addresses for symbol IDs are contained in the firmware image, the driver IDs will refer to any new addresses if the image is updated.

```
/**
 * Writes a firmware control corresponding to the respective symbol_id.
 *
 */
uint32_t regmap_write_fw_control(regmap_cp_config_t *cp, fw_img_info_t *f, uint32_t symbol_id,
uint32_t val)
{
    uint32_t temp_reg_addr, ret;

    temp_reg_addr = fw_img_find_symbol(f, symbol_id);

    if (temp_reg_addr == 0)
    {
        return REGMAP_STATUS_FAIL;
    }

    ret = regmap_write(cp, temp_reg_addr, val);

    if (ret)
    {
        return REGMAP_STATUS_FAIL;
    }

    return REGMAP_STATUS_OK;
}
```

4.5 Symbol Header

Because the symbol linking table contributes to the overall size of the image, it is beneficial to restrict the symbols to only the ones used by the driver, in order to optimize the image size. When using the fw_img_v2 format, a symbol header file can be provided to specify the required registers and their IDs.

The symbol header informs the firmware converter which registers to find addresses for and their place in the symbol linking table. The driver will also reference the symbol header for the IDs used by each register in the table.

Symbol headers are included as a part of the SDK in the device 'config' directory. When using the fw_img_v2 format, the symbol header is provided as an input using the --sym-input option.

4.6 Driver Loading Example - fw_img_v2

Firmware loading occurs in the application layer. In the CS40L50 SDK sample applications, this occurs in cs40l50/bsp/bsp_cs40l50.c in the bsp_dut_boot() function. The BSP uses code in common/fw_img.c/.h.

```
uint32_t bsp_dut_boot(void)
{

    uint32_t ret;
    const uint8_t *fw_img;
    const uint8_t *fw_img_end;
    uint32_t write_size;

    fw_img = cs40l50_fw_img;
...
```

5 Revision History

Revision	Changes
R1 FEB 2025	• Initial release
R2 JAN 2026	• Added *--binary output* option

Contacting Cirrus Logic Support

For all product questions and inquiries, contact a Cirrus Logic Sales Representative.
To find the one nearest you, go to www.cirrus.com.